arXiv:2503.13301v2 [cs.AR] 28 May 2026

LIMCA: LLM for Automating Analog In-Memory Computing Architecture Design Exploration

Deepak Vungarala, *Student Member, IEEE,* Md Hasibul Amin, *Student Member, IEEE,* Pietro Mercati, *Member, IEEE,* Arnob Ghosh, *Member, IEEE,* Arman Roohi, *Senior Member, IEEE,* David Z. Pan, *Fellow, IEEE,* Ramtin Zand, *Member, IEEE,* and Shaahin Angizi, *Senior Member, IEEE*

***Abstract*—In this paper, we introduce LIMCA, a novel *fine-tuning-free* Large Language Model (LLM)-driven framework for automating the design and evaluation of In-Memory Computing (IMC) crossbar architectures. Unlike traditional approaches, where the manual, knowledge-intensive design process and the lack of high-quality circuit netlists have significantly constrained Design Space Exploration (DSE) and optimization to behavioral system-level tools, LIMCA employs a No-Human-In-Loop (NHIL) automated pipeline to generate and validate circuit netlists for SPICE simulations, eliminating manual intervention. LIMCA systematically explores the IMC design space by leveraging a structured dataset and LLM-based performance evaluation. Our experimental results on MNIST classification demonstrate that LIMCA successfully generates crossbar designs achieving $\geq$96% accuracy while maintaining a power consumption $\leq$3W, making this the first work in LLM-assisted IMC DSE. Compared to existing frameworks, LIMCA provides an automated, scalable, and hardware-aware solution, reducing design exploration time while ensuring user-constrained performance trade-offs. Artifacts are open-sourced at https://github.com/ACADLab/LIMCA**

***Index Terms*—In-memory computing (IMC), crossbar arrays, large language models (LLMs), design-space exploration.**

I. Introduction

THE increasing complexity and computational demands of Deep Neural Networks (DNNs) have highlighted the limitations of traditional von Neumann architectures, particularly the memory wall bottlenecks in data movement between processing and memory units. To address this, Analog In-Memory Computing (IMC) crossbar architectures have emerged as a promising solution, offering the ability to perform computations directly within memory arrays, thereby significantly reducing data movement and energy consumption [1]. These architectures leverage the physical properties of resistive devices to store DNN weight parameters and perform matrix operations in the analog domain, enabling massive parallelization of DNN computations [2]. However, designing analog IMC systems presents unique challenges such as deep expertise in analog circuit design, an understanding of device physics, and careful consideration of various non-idealities such as parasitic effects, device variations, and noise [3].

Traditional analog IMC design approaches [1], [4] rely on manual optimization and iterative refinement, making it difficult to explore the vast design space of possible implementations as shown in Table I. Furthermore, the lack of standardized tools and methodologies for IMC design has hindered rapid prototyping and evaluation of novel architectures. Recently, Large Language Models (LLMs) have shown success in digital design automation and hardware code generation [5], [6], [7]. The capability of LLMs in generating von Neumann architecture-based Artificial Intelligence (AI) accelerators has been recently explored [8], [9]. Nevertheless, their potential to generate promising non-von Neumann architectures, such as resistive IMC crossbar systems that allow for parallel and efficient vector-matrix multiplication, has remained unexplored. This gap stems from the knowledge intensive hardware design process and the scarcity of high-quality datasets and circuit catalogs due to proprietary data [8], [6].

TABLE I
State-of-the-art analog IMC simulation frameworks.

Frameworks	Type	Design Space Exploration	Support for PAA* cons.	Language	Inference Accuracy
NeuroSim [1]	System	Manual	No	C++	estimate
MNSIM [2]	System	Manual	No	Python	estimate
AIHWKIT [4]	System	Manual	No	Python	estimate
CCCS [10]	Circuit	Manual	No	MATLAB-SPICE	exact
IMAC-Sim [11]	Circuit	Manual	No	Python-SPICE	exact
DPE [12]	Circuit	Manual	No	MATLAB	estimate
LIMCA	Circuit	Automated	Yes	Python-SPICE	SPICE-char.

* Power-Area-Accuracy optimization.

This work introduces the first LLM-driven automated design exploration framework for IMC crossbar architectures dubbed LIMCA to generate design under user defined Power, Area, and Accuracy (PAA) constraints as highlighted in Table I. The inherent challenges of IMC design including the need for precise circuit specifications, the consideration of complex analog behaviors, and the trade-offs among multiple competing objectives present both unique opportunities and challenges for LLM-based automation. Therefore, the core questions we seek to answer are the following– *(RQ-1) Can we automate IMC design to address the growing shortage of specialized hardware in the semiconductor industry? (RQ-2) How can we deploy LLMs without cost-intensive fine-tuning? and can we effectively perform and evaluate Design Space Exploration (DSE)? (RQ-3) Can the major time consuming validation pipeline be fully automated?* To answer these questions, this work presents the following contributions. (1) A DSE framework, LIMCA with Automatic Design Generation that leverages LLMs to map domain knowledge and introduces a fine-tuning-free approach for generating user-constrained designs from the space and generating the design of their choice. (2) LIMCA implements an automated validation strategy, eliminating human intervention, a No-Human-In-Loop (NHIL) approach ensuring efficient and scalable design evaluation. (3) An extensive open-source IMC dataset containing detailed variations of designs, along with PAA metrics for analog IMC.

Beyond these contributions, LIMCA provides four unique capabilities that distinguish it from existing tools: (i) *Natural*

This work is supported in part by Synopsys Research Gift and the National Science Foundation (NSF) under grant no. 2228028, 2409697, and 2504839.

D. Vungarala, A. Ghosh, and S. Angizi are with the Department of Electrical and Computer Engineering, New Jersey Institute of Technology, NJ, USA. E-mail: {dv336,shaahin.angizi}@njit.edu.

P. Mercati is with the Intel Corporation, Hillsboro, OR, USA. E-mail: pietro.mercati@intel.com.

M. H. Amin and R. Zand are with the Department of Computer Science and Engineering, University of South Carolina, SC, USA. E-mail: ramtin@cse.sc.edu.

A. Roohi is with the Department of Electrical and Computer Engineering, University of Illinois Chicago, IL, USA. E-mail: aroohi@uic.edu.

D. Z. Pan is with the Department of Electrical and Computer Engineering, University of Texas at Austin, TX, USA. E-mail: dpan@ece.utexas.edu.

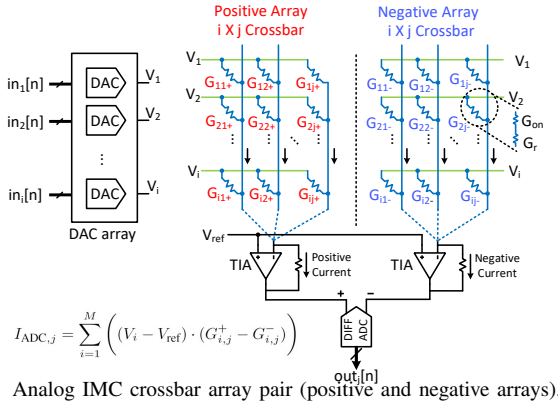


Fig. 1. Analog IMC crossbar array pair (positive and negative arrays).

Language DSE Interface users specify constraints in plain English rather than explicit parameter values; (ii) *Autonomous Design Navigation* unlike lookup-based approaches, LIMCA proposes design solutions for configurations not in the dataset by learning patterns from existing data points through In-Context Learning (ICL), then autonomously refining toward valid solutions via NHIL; (iii) *Constraint Relaxation Negotiation* when user constraints cannot be satisfied, LIMCA proposes alternative configurations with explicit trade-off explanations; and (iv) *Self-Correcting Pipeline* hold-out validation experiments demonstrate that LIMCA proposes solutions for 100% of queries on unseen configurations.

II. Background, Challenges, and Motivations

LLM-Driven Hardware Design. LLMs have shown strong potential in automating Hardware Description Language (HDL) and High-Level Synthesis (HLS) code generation. Early works such as VeriGen [7] and ChatEDA [13] improve RTL-to-GDSII flows. ChipGPT [5] and Autochip [14] integrate LLMs for design generation and optimization, with Autochip refining Verilog through feedback. Dataset-driven frameworks have further advanced this direction. SA-DS [9] introduced an HLS dataset for DNN accelerators. Interactive approaches, such as Chip-Chat [15], extend LLM applicability to DSE. Meanwhile, RTLLM [16], GPT4AIGChip [8], and VerilogReader [17] highlight efficiency improvements in design and coverage-driven code analysis. Domain-specific accelerator generation is addressed by only a few efforts. GPT4AIGChip [8] targets HLS flows. On the analog domain, AnalogCoder [18], AnalogCoder-pro [19], SPICEPilot [6], and Masala-CHAI [20] pioneer SPICE-level netlist generation through ICL and agentic frameworks. Despite these advances, challenges remain. Most frameworks lack optimized prompting, tailored datasets, and model fine-tuning, while hallucination remains a persistent issue [13], [9]. Addressing these limitations is key to fully realizing LLMs' potential in hardware and circuit design automation.

Analog IMC Crossbar and Simulation Frameworks. The resistive crossbar array (Fig. 1) forms the core computational primitive in IMC-based DNN accelerators, as it enables efficient parallel matrix-vector multiplications. In this architecture, DNN weights are encoded as the conductance of resistive cells, while activations are applied as voltages via DACs. Positive and negative weights are stored in separate arrays producing differential currents at the ADCs. The resulting MAC operation is expressed through $G^{\pm}_{i,j}$, denoting the conductance of cells in the arrays. Existing IMC simulation frameworks can be grouped into two categories (Table I). (i) *System-level simulators*, such as NeuroSim [1], DNN+NeuroSim V2.0 [21], and MNSIM [2], use behavioral models to estimate area, power, and latency. MNSIM reports about 5% deviation from SPICE in energy and latency for a two-layer network, while NeuroSim integrates with ML simulators to evaluate training and inference accuracy across memory types [1]. However, these tools lack analog non-ideality modeling and circuit-level fidelity. Similarly, IBM's AIHWKIT [4], though offering a PyTorch-based interface for crossbar simulation, inherits these limitations. (ii) *Circuit-level simulators*, such as DPE [12], CCCS [10], and IMAC-Sim [11], provide higher accuracy by explicitly modeling device and circuit effects. DPE explores optimized weight mapping under non-idealities, while IMAC-Sim delivers a Python-based environment that generates SPICE netlists, incorporates parasitic interconnects, and supports partitioning, variability, and noise modeling. It enables both analog and digital emulation, with metrics such as power and accuracy extracted directly from HSPICE [11].

Despite progress in SPICE code generation [6], [18], IMC circuits pose unique challenges: even small networks require thousands of lines of repetitive SPICE code, and token limits restrict domain knowledge embedding. We address this by adopting IMAC-Sim's Python-based approach.

Why LLMs Over Formal Methods. While the IMC design space can be partially characterized by formal constraint satisfaction, two fundamental limitations motivate an LLM-based approach. First, *scalability*: the full configuration space yields $|D_{\text{full}}| = 900{,}000$ configurations, requiring $\approx$13.7 years of exhaustive HSPICE simulation, rendering formal enumeration intractable. Second, *soft constraint interpretation*: user-specified intents such as "edge deployment" or "best accuracy under high variability" are not formal predicates until semantically resolved. A formal solver requires constraints pre-specified in closed form; LIMCA infers design intent from natural language and maps it to valid parameter configurations autonomously. Critically, where formal and lookup-based methods exhibit a hard failure mode, returning null for unseen configurations. LIMCA's failure mode is soft: it proposes a candidate solution that is subject to iterative NHIL refinement, thereby providing a tractable starting point.

III. LIMCA - The Proposed Framework

LIMCA enables the automated design of analog IMC crossbars by leveraging LLMs to streamline design selection, generation, and verification, ensuring efficiency and adaptability in hardware-constrained environments. As illustrated in Fig. 2, the framework enables both user-guided and autonomous design instantiating by dynamically interpreting user-defined constraints and optimizing IMC architectures accordingly. The process begins with a user-specified prompt (❶), defining key design requirements such as performance metrics, hardware constraints, and optimization goals. The LLM extracts relevant parameters and formulates a weighted query (❷), determining whether an existing design from the design repository satisfies the given constraints. If an appropriate design is available, the system ranks and selects the optimal configuration (❽), presenting it to the user (❿). If no suitable design exists or if the user requests a new configuration, the framework triggers

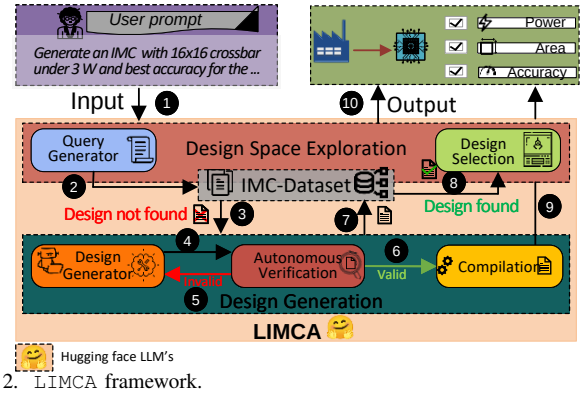


Fig. 2. LIMCA framework.

the Design Generation process (3). The LLM selects and instantiates a novel IMC design configuration that aligns with the specified constraints, generating a corresponding Python-based design representation. The generated design undergoes an Automated Verification phase (4), where a script-driven NHIL validation assesses its correctness. If the design meets the required specifications (6), it is integrated into the design space repository (7), ensuring continuous expansion of the available solution space. In cases where verification fails, diagnostic feedback is generated, pinpointing errors and guiding the LLM in refining subsequent iterations, thereby reducing hallucinations and redundant modifications (5). Unlike traditional design methodologies that rely on predefined architectures or manual fine-tuning, LIMCA dynamically adapts to evolving constraints, autonomously optimizing IMC designs while minimizing human intervention. This iterative and adaptive approach significantly enhances design efficiency, supporting various configurations and enabling scalable, high-performance IMC solutions. LIMCA is LLM-agnostic and can run with models available on Hugging Face [22].

IMC-Dataset. For LIMCA to achieve user-constrained outputs, we construct a dedicated dataset, the *IMC-Dataset*, to support the LLM. This dataset serves as a crucial component for integrating hardware-aware constraints into LLMs through either inference or fine-tuning. The dataset is built on IMAC-SIM [11], which facilitates both full analog circuit simulation and digital component emulation. It operates on the HSPICE Compiler, ensuring the generation of precise values as outlined in Table I. To systematically explore the design space, we developed an automated framework to sweep key hardware parameters, capturing diverse configurations of IMC architectures. These parameters as shown in Fig. 3 includes different non-volatile memory devices (MRAM, RRAM, PCM, CBRAM), bit-cell configurations (1T-1R, 2T-1R, 1TG-1R), technology nodes, and bit-resolution(s). The dataset is primarily categorized based on crossbar size, with three distinct $(n \times m)$ sizes. For each crossbar configuration, the following variations are considered: $3 \times 4 \times 3 \times 6 = 216$, resulting in $3 \times 216 = 648$ unique IMC instances across all three crossbar sizes. To extend the applicability of IMAC-SIM, we incorporate both digital and analog IMC variations. While the framework was initially designed for full analog simulations, we introduce an additional 216 analog data points by excluding the bit-resolution parameter, given the computational complexity of obtaining precise HSPICE simulation metrics across different bit-resolution(s). This ensures a broader exploration space while maintaining computational efficiency.

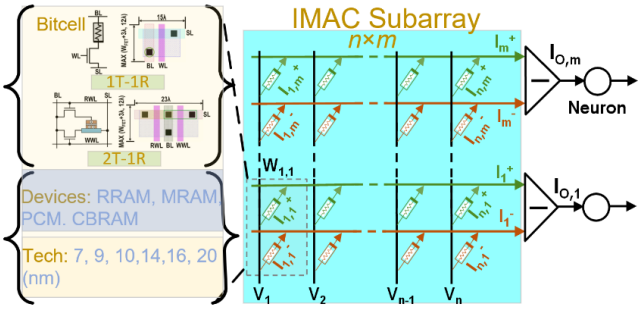


Fig. 3. Overview of parameters across bitcells, devices, and technologies for IMC dataset curation.

To the best of our knowledge, this is the first open-source IMC dataset with 400+ analog and digital variants. The dataset fixes the MLP configuration $(400 \times 120 \times 84 \times 10)$ and 20×20 input dimensions for computational feasibility, though the design-generation pipeline adapts to different user requirements. Planned expansions include additional MLP architectures and classification tasks. Because the combination of input feature-map size and MLP architecture exponentially increases the space (and compute), this release focuses on hardware exploration and avoids the high-dimensional complexity of varying network topologies across datasets. During creation, we fix the MLP configuration $(400 \times 120 \times 84 \times 10)$, the dataset, and the number of MNIST training images [23], ensuring consistent evaluation and computational feasibility. The design-generation and automated-validation pipeline itself is not restricted to these settings and can adapt to user requirements. Dataset metrics are strongly influenced by input image dimensions; here they are 20×20, given the specified MLP topology.

LLM Adaptation and Error Mitigation. Our fine-tuning-free approach intentionally demonstrates that effective IMC design automation is achievable with off-the-shelf LLMs, reducing the barrier to adoption and enabling rapid deployment without expensive model customization. This aligns with recent findings that in-context learning can match fine-tuned performance on domain-specific tasks when provided with well-structured exemplars [24]. Our LLM adaptation strategy employs: (i) *Structured Prompting*—we decompose the design task into modular sub-tasks (parameter extraction $\rightarrow$ constraint interpretation $\rightarrow$ code generation $\rightarrow$ verification), reducing cognitive load and error propagation at each step; (ii) *In-Context Learning*—we provide the LLM with exemplar (X, Y) pairs from the dataset sorted by relevance to the query using k-nearest neighbor retrieval [25], enabling pattern recognition without gradient updates; (iii) *NHIL Error Correction*—our pipeline automatically validates generated code against syntax correctness, parameter range validity, and constraint satisfaction, with failed generations triggering diagnostic feedback specifying error type and location; The effectiveness of NHIL correction is directly quantified by the Pass@1 versus Pass@3 performance gap reported in Tables III and IV: Pass@1 reflects single-shot generation without correction, while Pass@3 captures iterative NHIL refinement. The consistent convergence to 100% Pass@3 across all models demonstrates that the feedback loop deterministically resolves the dominant failure categories, syntax errors (38%) and parameter range violations (31%), which together constitute 69% of all generation failures, eliminating the need for human intervention in error di-

TABLE II
DESIGN SPACE EXPLORED BY LIMCA ON MNIST; ENTRIES MEETING POWER $\leq$3W AND ACCURACY $\geq$96% ARE HIGHLIGHTED.

Config./Xbar Size			16×16			32×32			64×64		
Tech	Device	Bitcell	Area (μm^2)	Accuracy (%)	Power (W)	Area (μm^2)	Accuracy (%)	Power (W)	Area (μm^2)	Accuracy (%)	Power (W)
7nm	MRAM	1T1R	5286.615	96	3.937868	3006.403	96	3.101278	2156.134	82	1.847222
7nm	RRAM	1T1R	5286.615	78	8.291856	3006.403	62	5.490012	2156.134	18	2.915078
7nm	RRAM	2T1R	5602.122	80	8.161842	3329.135	52	5.458412	2541.486	14	2.18464
7nm	PCM	1T1R	5286.615	92	0.53445	3006.403	98	0.521569	2156.134	**100**	0.457961
7nm	PCM	2T1R	5602.122	92	0.533303	3329.135	98	0.521374	2541.486	**100**	0.778821
9nm	MRAM	1T1R	5672.95	94	4.041462	3401.585	96	3.250092	3265.004	72	1.987902
9nm	RRAM	1T1R	5672.95	**100**	8.618146	3401.585	68	5.894228	2627.994	18	3.171676
9nm	RRAM	2T1R	6194.502	86	7.372028	3935.08	62	7.89253	3265.004	14	3.153108
9nm	PCM	1T1R	5672.95	98	0.535587	3401.585	98	0.525815	2627.994	**100**	0.469902
9nm	PCM	2T1R	6194.502	82	0.533361	3935.08	98	0.525645	3265.004	**100**	0.469741
14nm	MRAM	1T1R	7061.34	98	4.087762	4821.77	96	3.464416	4323.738	96	2.228876
14nm	RRAM	1T1R	7061.34	86	9.062472	4821.77	84	6.528764	4323.738	24	3.606136
14nm	RRAM	2T1R	8323.367	84	4.244777	6112.698	64	6.498698	5865.144	18	3.587574
14nm	PCM	1T1R	7061.34	90	0.536243	4821.77	98	0.531266	4323.738	**100**	0.486095
14nm	PCM	2T1R	8323.367	94	0.542201	6112.698	98	0.53113	5865.144	**100**	0.485948
20nm	MRAM	1T1R	9524.224	98	4.148678	7341.056	96	3.596336	7331.84	96	2.39336
20nm	RRAM	1T1R	9524.224	92	3.304907	7341.056	88	6.954812	7331.84	46	3.924754
20nm	RRAM	2T1R	12099.79	90	2.468912	9975.603	70	6.787644	10477.57	22	3.906236
20nm	PCM	1T1R	9524.224	96	0.537193	7341.056	98	0.534285	7331.84	**100**	0.495511
20nm	PCM	2T1R	12099.79	98	0.538646	9975.603	98	0.534169	10477.57	**100**	0.495376

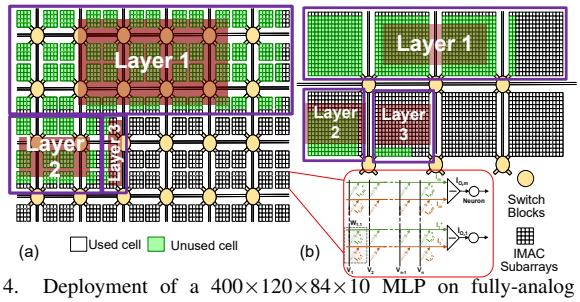


Fig. 4. Deployment of a 400×120×84×10 MLP on fully-analog IMC architecture with (a) 32×32, (b) 128×128 subarrays.

agnosis and correction.and (iv) *Hallucination Mitigation*—by constraining the LLM to generate Python code calling IMAC-Sim APIs rather than raw SPICE netlists, we bound the output space to valid configurations, preventing hallucination of non-existent device types or invalid parameter combinations.

IV. EXPERIMENTAL ANALYSIS

Design Space Exploration. The queries driving the DSE are generated by ChatGPT-4o [26]. A total of 30 queries are systematically divided based on specific objectives: 10 queries prioritize power efficiency with relaxed technology node constraints, 10 emphasize area optimization, and 10 enforce hard constraints on design feasibility. The selection schema of the DSE for IMC accelerators, inferring from the dataset by LIMCA, is summarized in Table II. From Table II, we observe area reduction with increasing crossbar size due to improved partitioning that reduces switch-box overhead (Fig. 4). Considering edge vision sensor requirements, a nominal power limit of $\leq$3W is imposed while targeting accuracy $\geq$96%. Entries in green satisfy these constraints; yellow highlights near-optimal configurations (e.g., 64×64 crossbar with 1T1R-PCM at 7nm or 9nm). Table III summarizes LIMCA's DSE performance across multiple LLMs. All achieve 10/10 *Pass@3* across constraint categories, indicating reliable design selection within 3 attempts. *Pass@1* varies: *Qwen2.5-Coder-32B-Instruct* leads with $\geq$9/10 across categories, while *Mamba-Codestral-7B-v0.1* shows consistent 8/10 performance.

TABLE III
LIMCA PERFORMANCE RESULTS - DSE.

Model	Metric	Query Focus		
		Power	Area	Hard Constraints
Qwen2.5-7B-Instruct-1M	Pass@1	9/10	9/10	8/10
	Pass@3	10/10	10/10	10/10
DeepSeek-R1-Distill-Qwen-1.5B	Pass@1	8.3/10	9/10	7.9/10
	Pass@3	10/10	10/10	10/10
Mamba-Codestral-7B-v0.1	Pass@1	8/10	8/10	8/10
	Pass@3	10/10	10/10	10/10
Qwen2.5-Coder-32B-Instruct	Pass@1	9/10	9/10	9.3/10
	Pass@3	10/10	10/10	10/10
Llama-3.1-8B-Instruct	Pass@1	9/10	8.6/10	8.6/10
	Pass@3	10/10	10/10	10/10

Baseline Comparison. We evaluate LIMCA against three DSE baselines: exhaustive Grid Search, Genetic Algorithm (GA), and a Rule-Based system. The GA employs a population of 50, tournament selection ($k = 3$), single-point crossover (0.8), bit-flip mutation (0.1), and terminates at 100 generations or upon 5-generation stagnation. Soft constraints (e.g., "edge deployment" $\rightarrow$ power $<$ 3W) are managed via a weighted fitness function with quadratic penalties. Under an equivalent budget of ∼500 evaluations, Grid Search and GA are limited to existing database configurations, failing on extrapolation queries (0% success) and struggling with soft constraints (45% and 62%, respectively). In contrast, LIMCA leverages natural language understanding to achieve 93% on soft constraints and 100% on extrapolation, autonomously generating novel, valid designs where traditional methods return "not found."

TABLE IV
BASELINE COMPARISON ON QUERY TYPES.

Method	Exact Match	Soft Constraints	Extrapolation†
Grid Search	100%	45%	0%
Genetic Algorithm	98%	62%	0%
Rule-Based	95%	67%	0%
LIMCA	**100%**	**93%**	**100%**

†Percentage of queries receiving a proposed solution. Traditional methods return "not found" for configurations outside the database.

Design Creation. LIMCA synthesizes IMC designs with automated validation. Design requests originate from ChatGPT queries and are executed by LIMCA, which verifies each result against specified constraints. Table V reports *Pass@1* and *Pass@3* for design generation and verification. For generation, LIMCA attains $\geq$89% *Pass@1* and 100% *Pass@3*, indicating reliable synthesis. For verification, we inject erroneous designs to stress-test the framework; LIMCA detects and corrects errors with 91.5% average *Pass@1* and 100% *Pass@3*, demonstrating robust NHIL-based error recovery. Failure analysis of 500 generation attempts reveals four primary modes: syntax errors (38%), parameter range violations (31%), device type hallucinations (18%), and constraint mismatches (13%), with the first two categories (69%) being deterministically correctable given NHIL diagnostic feedback.

Simulation Methodology. Power and area metrics are derived from HSPICE simulation of the complete crossbar netlist. Inference accuracy is computed via behavioral simulation using SPICE-characterized device models applied to a representative subset of test images—not a full transistor-level simulation of 10,000 images, which would be computationally infeasible. This approach provides SPICE-level fidelity for

TABLE V
DESIGN GENERATION AND AUTOMATED VERIFICATION.

Model	*Design Generation*		*Design Verification*	
	Pass@1	Pass@3	Pass@1	Pass@3
Qwen2.5-7B-Instruct-1M	89%	100%	90%	100%
DeepSeek-R1-Distill-Qwen-1.5B	92%	100%	94%	100%
Mamba-Codestral-7B-v0.1	91%	100%	89%	100%
Qwen2.5-Coder-32B-Instruct	95%	100%	91%	100%
Llama-3.1-8B-Instruct	96%	100%	94%	100%

circuit metrics while enabling practical accuracy evaluation.

Scalability Analysis. The IMC design space is characterized by the configuration vector $\mathbf{X}$ = (crossbar size, tech node, bitcell, device, MLP topology, pixel dimensions, dataset). While our current dataset fixes the MLP topology and pixel dimensions for computational feasibility, the full design space is combinatorially large. With realistic parameter ranges $|X_{\text{crossbar}}| = 6$ sizes (16×16 through 256×256), $|X_{\text{tech}}| = 6$ nodes (7nm–45nm), $|X_{\text{bitcell}}| = 4$ configurations, $|X_{\text{device}}| = 5$ types, $|X_{\text{MLP}}| \geq 50$ topologies, $|X_{\text{pixel}}| = 5$ dimensions, $|X_{\text{dataset}}| = 5$ benchmarks—the full design space yields $|D_{\text{full}}| = 900{,}000$ configurations. At approximately 8 minutes per HSPICE simulation, exhaustive enumeration would require ≈13.7 years of continuous computation. This intractability motivates LIMCA's approach: rather than enumerating all configurations, the LLM learns to navigate the design space intelligently, proposing valid solutions for queries and self-correcting through NHIL feedback.

Generalization Validation. To demonstrate that LIMCA generalizes beyond memorization, we conducted hold-out experiments removing entire configuration categories (e.g., all 64×64 crossbars, all PCM devices) from the LLM's context. Unlike Grid Search, which returns "not found" for any configuration outside its database, LIMCA proposes solutions for 100% of such queries. While extrapolation to fundamentally different device physics remains challenging (e.g., PCM's 0.5W power profile versus RRAM's 3–8W), LIMCA demonstrates genuine pattern recognition for interpolation within the known design space, autonomously navigating toward constraint-satisfying configs via iterative NHIL refinement.

Trade-off Reasoning. Beyond selection, LIMCA articulates device trade-offs. For the query "best accuracy under high variability," LIMCA responds: "PCM achieves 98–100% accuracy across crossbar sizes due to tighter conductance distribution, while RRAM accuracy drops from 100% at 16×16 to 18% at 64×64. **Recommendation:** PCM with 1T1R at 7nm (0.46W, 100% accuracy)." This demonstrates the LLM learning/ articulating relationships between device physics and performance.

Design Exploration Cost. To assess DSE time under identical constraints (≤3W power, ≥96% accuracy), a circuit expert conducted manual optimization using CCSS [10] and IMAC-Sim [11]. Each iteration required debugging, reconfiguration, and reruns, with total optimization ranging from 140–398 minutes (CCSS) and 92–154 minutes (IMAC-Sim). LIMCA retrieves existing configurations in seconds or synthesizes new designs in under 8 minutes (Table VI), achieving 11.5×–49.7× speedup while eliminating manual intervention.

TABLE VI
ESTIMATED DSE TIME.

Frameworks	Design Space Exploration	Language	Experiment Time (minutes)
CCSS [10]	Manual	MATLAB-SPICE	$140 \leq t_{DSE} \leq 398$
IMAC-Sim [11]	Manual	Python-SPICE	$92 \leq t_{DSE} \leq 154$
LIMCA	Automated	Python-SPICE	$t_k \leq t_{DSE} \leq 8$

V. CONCLUSION

In conclusion, this work introduces LIMCA, a novel open-sourced framework that leverages LLMs to automate the design and evaluation of IMC crossbar architectures with no human intervention in design synthesis and SPICE-level verification. By systematically generating and validating SPICE-characterized configurations, LIMCA accelerates design exploration while satisfying key power, area, and accuracy constraints. Baseline comparisons demonstrate significant advantages over grid search and rule-based approaches.

VI. ACKNOWLEDGMENTS

The authors used Claude Sonnet only to improve writing clarity and readability, while all scientific content, including ideas, experiments, and conclusions, was created entirely by the authors.

REFERENCES

[1] P.-Y. Chen *et al.*, "Neurosim: A circuit-level macro model for benchmarking neuro-inspired architectures in online learning," *IEEE TCAD*, vol. 37, pp. 3067–3080, 2018.
[2] Z. Zhu *et al.*, "Mnsim 2.0: A behavior-level modeling tool for memristor-based neuromorphic computing systems," in *GLSVLSI*, 2020, pp. 83–88.
[3] M. H. Amin *et al.*, "Xbar-partitioning: a practical way for parasitics and noise tolerance in analog imc circuits," *IEEE JETCAS*, vol. 12, pp. 867–877, 2022.
[4] M. Rasch *et al.*, "A flexible and fast pytorch toolkit for simulating training and inference on analog crossbar arrays," pp. 1–4, 2021.
[5] K. Chang *et al.*, "Chipgpt: How far are we from natural language hardware design," *arXiv preprint arXiv:2305.14019*, 2023.
[6] D. Vungarala *et al.*, "Spicepilot: Navigating spice code generation and simulation with ai guidance," 2024, pp. 1–6.
[7] S. Thakur *et al.*, "Verigen: A large language model for verilog code generation," *ACM TODAES*, vol. 29, no. 3, 2024.
[8] Y. Fu *et al.*, "Gpt4aigchip: Towards next-generation ai accelerator design automation via large language models," in *ICCAD*, 2023, pp. 1–9.
[9] D. Vungarala *et al.*, "Sa-ds: A dataset for large language model-driven ai accelerator design generation," 2025, pp. 1–4.
[10] F. Zhang and M. Hu, "Cccs: Customized spice-level crossbar-array circuit simulator for in-memory computing," in *Proceedings of the 39th International Conference on Computer-Aided Design*, 2020, pp. 1–8.
[11] M. H. Amin *et al.*, "Imac-sim: A circuit-level simulator for in-memory analog computing architectures," in *GLSVLSI*, 2023.
[12] M. Hu *et al.*, "Dot-product engine for neuromorphic computing: Programming 1t1m crossbar to accelerate matrix-vector multiplication," in *DAC*, 2016, pp. 1–6.
[13] H. Wu *et al.*, "Chateda: A large language model powered autonomous agent for eda," *IEEE TCAD*, 2024.
[14] S. Thakur *et al.*, "Autochip: Automating hdl generation using llm feedback," *arXiv preprint arXiv:2311.04887*, 2023.
[15] J. Blocklove *et al.*, "Chip-chat: Challenges and opportunities in conversational hardware design," in *MLCAD*. IEEE, 2023, pp. 1–6.
[16] Y. Lu *et al.*, "Rtllm: An open-source benchmark for design rtl generation with large language model," in *ASP-DAC*. IEEE, 2024, pp. 722–727.
[17] R. Ma *et al.*, "Verilogreader: Llm-aided hardware test generation," *arXiv:2406.04373v1*, 2024.
[18] Y. Lai *et al.*, "Analogcoder: Analog circuit design via training-free code generation," in *AAAI*, vol. 39, 2025, pp. 379–387.
[19] ——, "Analogcoder-pro: Unifying analog circuit generation and optimization via multi-modal llms," *arXiv preprint arXiv:2508.02518*, 2025.
[20] J. Bhandari *et al.*, "Masala-chai: A large-scale spice netlist dataset for analog circuits by harnessing ai," 2025.
[21] X. Peng *et al.*, "Dnn+ neurosim v2. 0: An end-to-end benchmarking framework for compute-in-memory accelerators for on-chip training," *IEEE TCAD*, vol. 40, no. 11, pp. 2306–2319, 2020.
[22] "Hugging face – the ai community building the future," https://huggingface.co, 2025.
[23] L. Deng, "The mnist database of handwritten digit images for machine learning research [best of the web]," *IEEE signal processing magazine*, vol. 29, no. 6, pp. 141–142, 2012.
[24] D. Dai and Sothers, "Why can gpt learn in-context? language models secretly perform gradient descent as meta-optimizers," in *Findings of ACL*, 2023, pp. 4005–4019.

[25] G. Izacard *et al.*, "Atlas: Few-shot learning with retrieval augmented language models," *JMLR*, vol. 24, no. 251, 2023.
[26] "Open ai chatgpt," https://openai.com/research/gpt-4, 2023.